COL851/8590: Special Topics in Operating Systems

Classes in AA slot. Mondays and Thursdays 2-3:30 pm. LHC519.

Course Content

NOTE: Topics below are tentative (until we are past that week). We will update it as we go through the lectures in each week.

Lecture 1: ML inferences overview MLinferences_overview.pdf **Lecture 2:** ML Model Architecture changes to reduce computations and storage: (i) Original computations and storage needed in basic CNN forward pass cnn-basics.pdf, (ii) 1x1 bottleneck layer inception-1x1-bottlenecklayer.pdf, Andrew NG video, (iii) depthwise separable convolutions mobilenet-depthwise-separable-convolution.pdf **Lecture 3:** ML model compression techniques: (i) Quantization quantization.pdf, (ii) Pruning pruning.pdf **Lectures 4-5:** The difference between Gen1 and Gen2 NN compilers NNcompilers.pdf, Gen2 NN compiler which optimizes computations to utilize target hardware capabilities ANSOR_slides.pdf **Lectures 6-7:** Runtime management in an inference serving system, like scheduling and load balancing NNruntime.pdf	Additional online resources on ML model compression: tensorflow, tensorflowlite, ristretto, pytorch, STM32, nvidia tensorrt, tensorrt sample script
Necessary readings for quiz1: (i) Mobilenet paper, (ii) Scalpel paper , (iii) Compiler based optimizations of inference computations for target hardware with Ansor (iv) Runtime management of inference servers with Nexus	**Aug 18**

Transformer and LLM background: slides_aug25.pdf, jupyter_aug25.ipynb based on the online tutorial (prelude and part 3 on transformer) https://medium.com/@bradneysmith/88a1320e7650	Other online resources: Next word prediction task video1, video2, video3, video4 karpathy_video1, with git repo, karpathy_video2 Translation with transformer model umarjamil_video1, Coding transformer from scratch umarjamil_video2, LLama architecture and features umarjamil_video3, coding LLama from scratch umarjamil_video4
Flash Attention Rewriting the attention kernels, to use the Nvidia GPU hardware optimally. In Flashattention-1, smaller, faster SRAM is used as memory for attention computation instead of larger, slower HBM. To fit SRAM, computation is done in tiles. Main innovation is algebraic, showing tiled softmax is exactly same as non-tiled softmax. Fusing (matmul-softmax-matmul) operations in a single kernel, intermediate N*N outputs of attention scores S and Probability P, are not wriiten out/read from HBM. Smaller stats are saved in forward pass, enough for gradient to be **recomputed** in backward pass. Flashattention 2 and 3 optimizes how this tiled kernel is parallelized across thread blocks and warps, to better utilize upcoming hardware features like TMA, and tensor cores. Original papers by Tri Dao et. FlashAttention neurips22, FlashAttention-2 iclr24, FlashAttention-3 neurips24. Slides used in class slides1, slides2. Arunabh notes on flashattention-1 notes1, Sarthak notes on flashattention-2 notes2, notes3, additional flashattention-1 notes washington. Background material on Hopper/Blackwell GPU architectural extensions used in Flashattention-3 cuda-programming-guide sections 10.26 (warp specialization for producer consumer) and 10.29 (Tensor Memory Accelerator). Jacobian calculation for softmax softmax.pdf. **POD Attention** This also rewrites the attention kernels, to better use the Nvidia GPU hardware. Each SM has known number of hardware units to process load/store instructions (useful for decode stage of LLM) and compute instructions (useful for pre-fill stage of LLM). But the CTA and warp schedulers are black boxes in the NVIDIA driver, not allowing the programmer to schedule prefill-decode across LLM inference requests to balance the use within the SM hardware. Pod-attention is a beautiful paper, circumventing the blackbox driver, using the same kernel for prefix-decode, but within the kernel deciding between prefill vs. decode, depending on that particular SM's hardware utilization. paper, slides	Youtube videos: Flashattention-1, Flashattention-2, Flashattention-3-jayshah, Flashattention-3-tridao, Podattention
KV cache optimization Using paging principles from OS, PagedAttention paper reduced internal and external fragmentation of KV cache memory usage. Their solution was implemented in userspace, which needed attention kernels to be re-written to handle non-contiguous virtual memory accesses. Followup work vAttention paper used cuda virtual memory management APIs, so that virtual memory could be contiguous, and attention kernels could much easily use paging. Slides pagedAttention_slides.pdf, vAttention_slides.pdf. Another direction in KV cache optimization, especially to handle very long sequence lengths, has been to introduce strutcured sparsity. Systems paper FlashInfer paper handles these sparse attention kernel computations, using block sparse matrices.	Youtube videos: pagedAttention till 24:33 minutes.
Necessary readings for quiz2 (i) Chapters 3, 4, 5 from cudabook, (ii) How to optimally parallelize histogram (iii) Flash Attention 1 paper, (iv) Flash Attention 2 paper, (v) Flash Attention 3 paper, (vi) podAttention paper, (vii) pagedAttention paper, (viii) vAttention paper,	**Sep 25 2:15-3:15 pm**

Evaluation

- Paper discussion with Riju. Create quiz questions from that paper **[10 marks]**
- 3-4 quizzes, one after finishing each sub-topic **[40 marks]**
- Literature survey PDF report on a sub-topic (equivalent of major exam) **[10 marks]**
- Assignment (individual), project (in pair)**[40 marks]**

Assignments

- **Assignment 1 (5 marks)**: Run LLM inference with different available base models and datasets on laptop CPU.
- **Project (35 marks) to be done in pair**: Time-series data: gurgaon.csv, and patna.csv. Column 5 is calibrated Particulate Matter (PM 2.5) values downloaded from https://vayu.undp.org.in/, averaged over all sensors in the city over an hour, column 1 is hourly timestamp (the data is for 185 days between Jul-Dec 2024), columns 2, 3 and 4 are meteorological factors relative humidity, temperature and wind speed respectively, which possibly affect PM.

  Part 1 and 2 demos on Oct 13.
  Part 3 and 4 demos on Nov 3.
  Part 5 demos on Nov 27, 28.

  - **Part 1: Analyzing the data and the pre-trained forecasting model** Run the pre-trained time-series LLM model for forecasting from Amazon chronos, on the data of the two cities above for PM forecasting task, on your laptop CPU. Use only column 1 and column 5 for this part, ignoring the meteorological factors. Experiment in zero-shot mode (i.e. without any fine-tuning), with (i) diferent context lengths (i.e. how many days in the past is used in forecasting the future),(ii) different forecasting horizons (i.e. how much into the future you forecast) and (iii) different pre-trained model variants, based on the compute power of your laptops. For 24 hours horizon, plot average RMSE with different context lengths (2 days, 4 days, 8 days, 10 days, 14 days) for the biggest model-variant you can run. For 10 days context length, plot average RMSE with different horizons (4 hours, 8 hours, 12 hours, 24 hours, 48 hours). Add the plots in your report. Discuss which model variant, context length and horizon gives best average RMSE for the two cities. **[6 marks]**
  - **Part 2: Analyzing performance metrics** Build a measurement framework, e.g. using perftool, to visualize forecasting inference performance for part 1 on laptop CPU, including performance metrics like inference latency, throughput, CPU utilization, DRAM utilization, cache hits/misses, etc.. For a loop of continuously running forecasting inferences, show these performance metrics (e.g. using grafana) in real time. Also plot the metrics varying context length, forecasting window and model variant, and add the plots in your report. Discuss RMSE (measured in part 1) vs. performance (measured in part 2) trade-offs if any. **[4 marks]**
  - **Part 3: Does adding meteorological factors improve forecasting RMSE?** For the best set of hyper-parameters found in part 1 (model variant, context length, horizon), plot the CDF of RMSE to see the whole distribution, instead of just the average RMSE. Plot test day along x-axis, ground truth and forecasted PM, and RMSE along y-axis, and discuss if high RMSE on some test-days is related to sudden increase or decrease in PM, causing difficulty in forecasting. Now try and include the meteorological covariates, and see if RMSE improves. If you need forecasted values of the covariates to forecast PM, use ground truth values of the covariates from the csv (assuming we have perfect forecast for weather parameters). For covariate inclusion, dataset might have to be divided into train-test set, as catboost/xgboost for mapping covariates to PM will need training. Use 80-20 split, and make sure to compare RMSE with and without covariates on the same 20% test data, for apple-to-apple comparison. Plot performance metrics with covariates, and compare with the performance metrics measured without covariates, in part 2. **[7 marks]**
  - **Part 4: Checking performance on more constrained embedded hardware** Run forecasting with and without covariates on Raspberry PI and compare performance metrics w.r.t your laptop measurements. Pin the process to 1, 2, 3 ... cores of the PI, and see the effect of reducing CPU cores. Additionally measure CPU temperature every minute, running inferences continuouly for 30 mins. Include these plots in your report, with discussion. This is relevant for personal PM exposure apps, that might be available on Android phones.**[8 marks]**
  - **Part 5: Implementing an inference server** Implement a server-client program on your laptop, so that the client sends forecasting requests with different hyper-parameters (context lengths, horizons, model variant, with and without co-variate) to the server. The server runs the appropriate request and returns the forecasted PM value to the client. List all LLM optimizations covered in lectures, with paper references. Mention which optimizations are not applicable on the CPU server, which are applicable but cannot be implmented due to restrictions with pre-trained models, compilers etc., and try and implement the rest (you can add more optimizations, which were not discussed in class, but makes sense to you). Create a trace of 500 client requests, using the datasets. Measure performance metrics for the server to handle these 500 requests, where (a) client does not wait between sending two requests, (b) client genrates requests with Poisson distribution. Plot without any implemented optimization, and with your optimizations. Discuss the effect of each optimization on server performance. **[10 marks]**